Slide 1 — Title

Hello everyone. Every metro train like the one students ride across cities to colleges and universities carries an air compressor. It's not a glamorous piece of equipment, but it makes the compressed air that runs the doors, the brakes, the suspension, and the climate control in the cabins. So when it starts to go wrong, the first thing passengers actually notice usually isn't a mechanical fault at all. It's that the carriage is too warm, or the doors are working slow. And when it fails completely, the train stops where it is and comes out of service. Which on a metro line in a large city is a genuinely expensive and frustrating afternoon.

I worked with a public dataset from one of these compressors, on the Porto Metro. One and a half million sensor readings, taken every ten seconds, across seven months of ordinary service. And buried in that record are four air leaks that the operator wrote up in their own maintenance logs. So the question I set out to answer was simple to state: could the data have seen them coming?

Do not explain any of the four numbers along the bottom of the slide yet. They are there to be glanced at. You explain them properly on slide 5, where they mean something.

Slide 2 — The project in one slide

WALK LEFT TO RIGHT ALONG THE THREE BOXES AS YOU TALK.

Here is the shape of the weeks I spent with the data. I want to be upfront that the plan changed twice, and both times it changed because the data told me I was wrong rather than because I ran out of time.

Before I'd looked at anything properly, I had a theory about what the fingerprint of an air leak would look like, and I built the whole design around it. A couple weeks in, when I finally had the data clean enough to test that theory, it turned out to be wrong — and I'll show you exactly how on the next slide, because the reason it was wrong was one of the most interesting things I found. Then at the very end, the prediction model I'd been building failed a test I'd set for it in advance, so I shipped a simpler backup instead.

POINT AT THE DARK BOX IN THE MIDDLE.

What I ended up with has two halves. The first half spots a fault that's happening right now. The second half tries to warn you before it starts. Those sound like the same problem but they're really not, and only one of them is properly solved.

POINT AT THE FOUR NUMBERS. SLOW DOWN ON THE LAST ONE.

A million and a half raw readings get boiled down to about seventeen thousand analysable chunks, and the whole cleaning process takes sixteen seconds. The detector catches all four leaks while they're happening. But it only manages to warn me in advance about two of the four. That last number is the

honest one, and I'd rather put it on the second slide than bury it on the ninth.

Slide 3 — How this machine actually fails

To explain what I found, I need to tell you how one of these compressors normally behaves. It doesn't run continuously. It kicks on, pumps air until the tank is full, and shuts off again. Then pressure slowly bleeds away, and a couple of minutes later it kicks on again. That on-off-on pattern is what I'll call a cycle, and a healthy unit does about two and a half cycles per hour.

So my original theory was this. If there's a leak, air escapes faster, which means the compressor has to work harder to keep the tank full, which means the cycles should get longer and closer together. Measure the cycles and you'd see the leak. I thought that was airtight, and I built the entire project on it.

PAUSE HERE. THIS IS THE TURN.

When I finally counted them, there were just under eleven thousand cycles across the seven months. Three of them happened during a leak.

And the reason is almost embarrassing once you see it. During a bad enough leak, the compressor can never get the tank full, so it never reaches the point where it shuts off. It just runs continuously. A normal cycle lasts a bit over two minutes; the longest unbroken run during a leak was more than twenty-five hours. So the thing I'd chosen to measure doesn't just get rare during a failure, it stops existing. I'd built a project around a ruler that vanishes exactly when you need to measure something.

So I changed the unit of measurement, and once I did, the picture on this slide fell out of the data. There are three stages. A healthy compressor cycles about two and a half times an hour — that's the green bar. In the two days before a leak is reported, that rises to a bit over three, so it's cycling about twenty percent more often. That's the machine working harder to keep up, exactly as I'd expected. And then during the leak, it collapses to almost nothing, because it has stopped cycling and is just running flat out.

MOVE TO THE RIGHT-HAND PANEL. THIS IS THE PAYOFF — SLOW RIGHT DOWN.

Now here's the part that matters. The right-hand chart is duty cycle, which just means the share of time the compressor is running rather than resting. That is the obvious thing to watch. It's what I would have watched, it's what most monitoring systems watch, and it's the first thing anyone reaches for.

Healthy, it sits at fourteen percent. In the two days before a failure, it sits at fourteen percent. It does not move at all until the moment the fault actually arrives, and then it jumps to nearly a hundred.

Which means — and this is the finding — any alarm built on duty cycle has no early warning in it whatsoever. Not because it's badly tuned, but because the thing it's measuring genuinely doesn't

change until it's too late. If you want warning, you have to count how often the compressor is switching on and off. As far as I can tell, that hasn't been written down for this machine before.

Slide 4 — The computer found the fault on its own

A 2:00 · B 1:30

POINT AT THE SCATTER PLOT WHILE YOU SET THIS UP.

Before trusting any of that, I wanted a sanity check that didn't involve me. So I took the whole seven months, stripped out every label, every mention of when the failures happened, and asked an algorithm to just sort the data into groups by similarity. It has no idea what a fault is, and it's only looking for clumps.

It came back with two groups, very confidently. The big one is ninety-four percent of the time: the compressor cycling on and off normally. The small one is the other six percent, and in that group the compressor is running essentially non-stop.

BEAT. THEN THE NUMBER.

Ninety-eight and a half percent of all the known failure time landed in that small group. So an algorithm that was never told failures exist went and found them anyway, which is about the best evidence I could ask for that the measurements I'd chosen really do capture the physics. It also took less than a tenth of a second to run, which I mention because almost nothing else in this project was that fast and cheap..

POINT AT THE AMBER BOX. THIS CAVEAT IS AS IMPORTANT AS THE FINDING.

But look at it from the other direction and there's a problem. Only about a third of that small group is a reported failure. The other two thirds is the compressor running flat out with no maintenance report attached to it at all. Now, either the operator didn't write everything down, or there are genuinely days when the service demands that much air. I can't tell which from the data, and nobody can.

What that means practically is that there's a ceiling on how accurate anything I build can look, and that ceiling is set by the quality of the record-keeping rather than by my modelling. Keep that in mind for the next slide, because it explains a number that would otherwise look disappointing.

Slide 5 — How well it works

A 3:15 · B 2:45

POINT AT THE BLUE GRID ON THE LEFT.

This is the scoreboard. On the left is every fifteen-minute slice in the second half of the data, sorted into whether there was really a fault and whether my system said so. Two numbers matter. Recall is how much of the real fault time I caught, and I caught ninety-eight and a half percent of it — five missed slices across five months. Precision is how often an alarm turns out to be right, and that's forty-one percent.

In plain terms: it essentially never misses a fault in progress, and it costs you about ninety nuisance alarms a month to get that.

POINT AT THE CREAM BOX. SAY THIS QUICKLY, IT'S AN ASIDE.

One thing I want to point out. My system is right ninety-six percent of the time. But a system that simply says ‘everything's fine’ forever, and never detects anything at all, is right ninety-seven percent of the time because faults are rare. So plain accuracy is worse than useless here. It actively rewards doing nothing, which is why you won't see it anywhere in my conclusions.

NOW THE UNCOMFORTABLE PART. DON'T RUSH IT, AND DON'T APOLOGISE.

Now, the honest comparison. Before building anything clever I tried the dumbest possible detector: one line of arithmetic that just asks whether the compressor is running right now. That gets a combined score of 0.577. Everything I built over eight weeks gets 0.580. The improvement is three thousandths.

So spotting a fault while it's happening is basically solved by the physics, and no amount of machine learning was going to add much. That's a real result and I'm going to report it as one rather than dress it up.

POINT AT THE BAR CHART. THIS IS THE REBUTTAL — IT MUST BE SAID.

Except that comparison isn't quite fair, and here's why. Every method gets to pick its own trigger point, and I let each one pick its best. That's not how a maintenance team works. They tell you how many false alarms a month they'll put up with — call it a budget — and then ask what you can deliver inside it. So this chart pins everybody to the same budget of ninety alarms a month and compares them there.

Judged that way, the gap isn't three thousandths, it's nearly nine hundredths. And notice the simple one-line detector isn't even on this chart, because it can't be dialled to a budget. It's either on or off. It has one setting, and if that setting doesn't match what your team can absorb, you're stuck.

So what the more complicated model actually buys you isn't accuracy. It's a dial. You can trade off how much you catch against how much noise you're willing to hear, and that turns out to be the thing that decides whether a system is deployable or not.

Slide 6 — Five months of it running

A 2:30 · B 2:00

ORIENT THEM BEFORE YOU INTERPRET ANYTHING. POINT AS YOU NAME EACH ELEMENT.

This is the finished system running across the whole second half of the data. The grey line is the compressor's workload. The red bands are the four leaks the operator documented. The orange squares along the top are the system saying ‘there's a fault right now.’ And the green triangles are it saying ‘I think one is coming.’

The detection part you can read at a glance. Every red band has orange sitting on top of it. Four out of four.

MOVE DOWN TO THE THREE ZOOMED PANELS.

The early warning is the other story, and it's a more mixed one. There are only five green triangles in five months. That's a very quiet system, which is good — it works out to less than one false alarm a month — but it's also exactly why it misses things. Being that reluctant to speak up means sometimes it doesn't.

The three panels underneath zoom in on individual events. The one on the right got ten and a half hours of warning, and you can see why: there's a visible ramp climbing towards the failure. The one in the middle got nothing at all, and you can see why there too — it's flat, and then it's vertical. There was no run-up for the system to notice.

THIS NEXT BIT PRE-EMPTS THE OBVIOUS CHALLENGE. KEEP IT.

I should be straight about something here, because it looks like a contradiction. Early on, before I had any of this working, I went back through the data by hand and found what looked like signs of trouble building sixty hours before that middle event — the one that got no warning at all. So why couldn't the finished system see it?

Because looking backwards from a failure you already know about is a completely different thing from spotting one coming. When you know the answer, you'll always find something that looks like a clue. The test that matters is how often that same clue appears on the thousands of days when nothing goes wrong, and I hadn't run it. So that sixty-hour figure was never a real warning. It was hindsight, and I'd rather say that plainly than let it stand.

Slide 7 — Fast enough to actually deploy

Quick practical point, because a monitoring system that can't keep up with its own data isn't a monitoring system. Start to finish — loading the raw file, cleaning it, and scoring every slice — the whole thing takes about sixteen seconds for seven months of data. Per fifteen-minute slice, that's about a millisecond, against fifteen minutes of real time to do it in. So it's roughly a million times faster than it needs to be.

And the slowest part isn't any of the clever bits. It's just reading the file off disk and lining the timestamps up. That's a plumbing problem, not a math problem, and it's the sort of thing that disappears in a production setup. The practical upshot is that this would run on a cheap computer bolted to the train itself. You don't need a data centre for this work.

Slide 8 — What worked and what didn't

LEFT COLUMN, BRISK. THESE ARE THE EASY WINS.

Three things worked. The grouping algorithm was far and away the best return on effort in the whole project — a tenth of a second of computing time, and it validated the entire approach. Cutting my measurements down from forty-one to fourteen, chosen because they meant something physically rather than because a computer liked them, improved results substantially. And refusing to paper over the gaps in the data mattered: there are nine hundred hours missing from these seven months, and I flagged every one rather than filling them in with plausible-looking guesses that would have taught the system that smooth fake data is what normal looks like.

RIGHT COLUMN. SLOW DOWN — THIS IS WHERE YOUR CREDIBILITY IS.

Three things didn't work. The first you've heard: the measurement I built the project around disappears during the exact events I was trying to study. And I want to be careful about the lesson, because the fix worked out well and it would be easy to claim I'd planned that, and I hadn't. It was a salvage job, and I got lucky that the replacement happened to carry the early signal. If this machine had failed slightly differently, the same fix would have given me nothing.

The second is that my sophisticated detector lost to that one line of math. I could have kept tuning it until it won and only shown you that version. Working out why it lost taught me more — the compressor simply works harder in summer than in winter, and the model was spending all its attention on the seasons instead of on the faults.

And the third one I still can't explain. I built ten new measurements aimed squarely at the early-warning signal I'd just discovered, measuring exactly the thing that should have helped, and they made the predictions slightly worse. With only four failures in the entire dataset, I genuinely cannot tell you why.

THE BOTTOM LINE. THIS IS THE PROCESS POINT — KEEP IT.

One last thing on that. Halfway through, before I knew how it would turn out, I wrote down a rule for myself: if the prediction model hadn't reached a certain standard by a certain date, I'd stop tuning it and ship the simple backup instead. It landed right on the line, close enough that the difference was inside the noise. So I took the conservative reading and shipped the backup. You can argue with that call. What I'd defend is having written the rule down before I knew the answer, because otherwise I'd still be tuning it.

Slide 9 — What someone should do next

A 1:30 · B 1:00

FOUR ITEMS, BUT THE ORDER IS THE POINT.

Four things, and I've deliberately ordered them by how much they'd actually change the answer rather than by how interesting they are.

First, and cheapest: just record when the compressor switches on and off. That's the signal that carries the early warning, and I had to reconstruct it indirectly from a sensor that wasn't designed to tell me. Logging it directly costs almost nothing and would sharpen everything.

Second, get more failures. Every limit I ran into came from having only four of them, not from the methods. There are sister datasets covering other trains and other kinds of fault, and pooling them together would help twice over — with more examples, and a test of whether any of this transfers.

Third, put it on a screen. The scoring is effectively free, so a live tableau dashboard or something similar for the maintenance desk is an easy build job, not a lengthy research job.

Fourth, try it on a different fleet, and find out whether these patterns are general or whether every compressor needs to learn its own habits.

LAND THIS LINE. IT'S THE THESIS OF THE WHOLE PROJECT.

And notice what the top of that list looks like. Two of the first three are about collecting better data, not about building better models. Multiple weeks of this told me quite firmly that the models were never the thing holding us back.

Slide 10 — What it all means

A 1:00 · B 0:55

NO NEW NUMBERS. THIS IS THE LANDING.

So, four things to take away. About the machine: it fails in three distinct stages, and the warning sign you'd naturally watch for only shows up once it's already too late to act.

About the modelling: the simplest thing I tried beat the most complicated thing I built. One line of arithmetic, based on nothing but how the machine physically works, outperformed forty-one measurements and a machine learning model. Understanding the equipment mattered more than the computing power every single time.

About the data: two thirds of the time this compressor was running flat out, and nobody wrote anything down. How good these systems can get is limited by the paperwork, not always by the algorithms.

And about the process: two things I planned failed and got replaced, and the reason the project finished at all is that I'd written down in advance what would count as giving up on one of them.

FINAL LINE. THEN STOP AND HOLD FOR A BEAT.

Where that leaves a maintenance team is this. They get a system that essentially never misses a fault that's happening, and a warning that arrives hours early about half the time. That's half the problem solved, and I'd rather hand over half a problem with the edges marked than a rounder number I couldn't stand behind. Thank you.

There is no Q&A. Do not end by offering to take questions — it invites silence. End on “thank you,” and stop.